

Lista 5 – Exclusão e Reaproveitamento de Registros e árvore B

Parte 1

Com base na lista 4, realize as ações necessárias para a exclusão e reaproveitamento de registros.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
5	M	A	R	I	A	5	R	U	A	b	1	4	123	10															
30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59
S	A	O	b	C	A	R	L	O	S	4	J	O	A	O	5	R	U	A	b	A	4								
60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
255	9	R	I	O	b	C	L	A	R	O	5	P	E	D	R	O	6												
90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119
R	U	A	b	X	V	4	56	10	S	A	O	b	C	A	R	L	O	S											
120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149
-1																													

1. Implementar a exclusão lógica de registros e a reutilização dinâmica de espaços em um arquivo estruturado, utilizando uma lista encadeada de registros removidos e o algoritmo Best Fit para alocação.

a) Exclusão Lógica de Registros

- Ao excluir um registro, substitua seu primeiro byte por * (marcando-o como logicamente removido).
- O registro não deve mais ser exibido em consultas.
- O **byte offset** (posição no arquivo) e o **tamanho do registro** devem ser inseridos em uma **lista encadeada de registros removidos**.

b) Armazenamento da Lista Encadeada

- A **cabeça da lista** deve ser armazenada no **cabeçalho do arquivo**.
- Cada entrada na lista deve conter:
 - **Byte offset** do registro removido.
 - **Tamanho do registro**.

- **Próximo nó** (byte offset do próximo registro removido ou -999 se for o último).

c) **Inserção com Reutilização (Best Fit)**

- Ao inserir um novo registro, **consulte a lista de removidos** e utilize o **algoritmo Best Fit** para encontrar o menor espaço disponível que comporte o novo registro.
- Se um espaço adequado for encontrado:
 - Remova-o da lista de removidos.
 - Reutilize o espaço para armazenar o novo registro.
- Caso contrário, grave o registro no final do arquivo.

Atenção: a lista encadeada de registros deve ser armazenada no mesmo arquivo de registros, conforme explicado em sala, elucidado nos slides. Ao ler o arquivo, você pode criar uma estrutura em memória principal, como uma lista encadeada (*linked list*) ou um *ArrayList*.

Parte 2

O livro "Algorithms, 4th Edition", escrito por Robert Sedgewick e Kevin Wayne e disponível no site algs4.cs.princeton.edu, é uma das principais obras globais sobre os fundamentos de Algoritmos e Estruturas de Dados. Adotado por diversas universidades em todo o mundo, o material se destaca por sua abordagem didática, precisa e voltada para a aplicação prática, tornando-se uma referência essencial para estudantes e profissionais da Ciência da Computação.

Um dos pontos fortes da obra é a integração entre teoria e prática, com exemplos de implementação em Java. Isso permite que os leitores apliquem os conceitos aprendidos diretamente na solução de problemas reais. A linguagem é utilizada de maneira pedagógica, com códigos organizados e bem documentados, facilitando o aprendizado mesmo para quem ainda está se familiarizando com programação orientada a objetos.

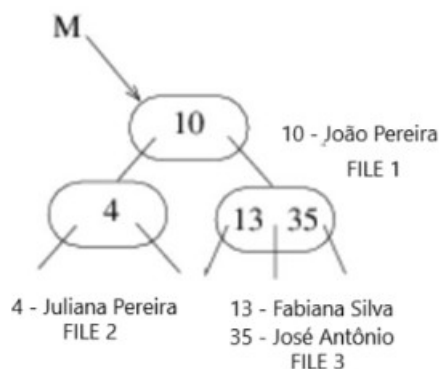
A estrutura do livro é progressiva, começando com temas introdutórios, como métodos de ordenação e busca, até chegar a tópicos mais complexos, como algoritmos em grafos, estruturas de dados avançadas e técnicas probabilísticas. Cada conceito é acompanhado por ilustrações, tabelas comparativas e análises de desempenho, ajudando os estudantes a desenvolverem uma compreensão crítica e aprofundada.

O site oficial complementa o conteúdo com exercícios, visualizações dinâmicas, códigos-fonte e conjuntos de dados reais, enriquecendo ainda mais o aprendizado. Além disso, o livro é referência em cursos online renomados, como os oferecidos por Princeton e Coursera, reforçando sua credibilidade no meio acadêmico.

1. Posto isso, na URL <https://algs4.cs.princeton.edu/code/> é apresentado o código-fonte de uma árvore B, com o nome BTree. Baixe o código, analise-o, compreenda seu funcionamento e faça as modificações necessárias para que o código possa ser executado no IntelliJ IDEA.

2. Modifique a classe da árvore B para que ela seja capaz de armazenar registros vinculados a arquivos descentralizados de um banco de dados. Para este caso, a árvore deve ser de ordem 2 (isto é, cada nó pode conter no máximo 4 chaves e 5 filhos). Adapte a estrutura para que cada entrada da árvore contenha uma chave de índice (por exemplo, um identificador numérico) e nome de usuários associados a essa chave. Atenção especial deve ser dada ao processo de overflow: ao ocorrer uma divisão de nós na árvore, os registros devem ser corretamente remanejados para os novos índices, com a devida atualização dos arquivos correspondentes a cada nó.

A seguir a um exemplo de ilustração para implementação.



Para descontrair

